

Foxconn Open BMC 新人訓

L4 - GPIO Configuration

Sonny Chu
2025/12/05

環境設定

系統資訊

項目	內容
作業系統	Ubuntu 22.04.5 LTS (VirtualBox)
核心版本	6.8.0-87-generic
Python	3.10.12
硬碟空間	1TB(約 750GB 可用)

OpenBMC 環境

項目	內容
OpenBMC 版本	2.18.0 (commit: 05f7a3f2ed)
MACHINE	romulus
原始碼路徑	~/openbmc
Build 路徑	~/openbmc/build/romulus
映像檔路徑	~/openbmc/build/romulus/tmp/deploy/images/romulus/

作業資訊

作業目標

建立一個 GPIO 監控服務(Monitor GPIO Service)，監控 GPIOA1(line 1)的狀態變化，當 GPIO 輸入狀態改變時(上升緣或下降緣)，輸出狀態變化訊息。

GPIO 配置

項目	設定值
GPIO 晶片	gpiochip0
GPIO Line	1 (GPIOA1)
Line 名稱	cfam-reset
模式	Input
監控事件	RISING_EDGE / FALLING_EDGE

檔案結構

檔案	說明
gpio-monitor.cpp	GPIO 監控服務主程式(使用 libgpiod)
meson.build	Meson 編譯設定檔
gpio-monitor.service	systemd 服務單元檔
gpio-monitor_1.0.bb	BitBake recipe 檔案

實作內容

Part 1: GPIO 環境確認

步驟 1: 查看 GPIO 控制器

目的: 確認 romulus 平台的 GPIO 控制器配置

gpiodetect

```
sonny@sonny-VirtualBox:~$ ssh -p 2222 root@127.0.0.1
root@127.0.0.1's password:
root@romulus:~# gpiodetect
gpiochip0 [1e780000.gpio] (232 lines)
root@romulus:~#
```

圖 1: gpiodetect 輸出 - gpiochip0 有 232 條 GPIO lines

備註: AST2500 的 Parallel GPIO 控制器記憶體映射地址為 0x1e780000, 共有 232 條 GPIO lines。

步驟 2: 查看 GPIOA1 資訊

目的: 確認 GPIOA1 對應的 line 編號和狀態

gpioinfo gpiochip0 | head -n 20

```
root@romulus:~# gpioinfo gpiochip0 | head -n 20
gpiochip0 - 232 lines:
   line   0:      unnamed      unused   input   active-high
   line   1: "cfam-reset"      unused   input   active-high
   line   2:      unnamed      unused   input   active-high
   line   3:      unnamed      unused   input   active-high
   line   4:      unnamed      kernel   input   active-high [used]
   line   5:      unnamed      kernel   input   active-high [used]
   line   6: "fsi-mux"      unused   input   active-high
   line   7:      unnamed      unused   input   active-high
   line   8:      unnamed      unused   input   active-high
   line   9:      unnamed      unused   input   active-high
  line  10:      unnamed      unused   input   active-high
  line  11:      unnamed      unused   input   active-high
  line  12:      unnamed      unused   input   active-high
  line  13:      unnamed      unused   input   active-high
  line  14:      unnamed      unused   input   active-high
  line  15:      unnamed      unused   input   active-high
  line  16:      unnamed      kernel   input   active-high [used]
  line  17:      unnamed      kernel   input   active-high [used]
  line  18:      unnamed      kernel   input   active-high [used]
root@romulus:~#
```

圖 2: gpioinfo 輸出 - line 1 為 GPIOA1, 名稱 "cfam-reset"

備註: GPIOA1 = line 1, 已有名稱 "cfam-reset", 目前為 input 模式且未被佔用。

步驟 3: 讀取 GPIOA1 初始值

目的: 確認 GPIOA1 的初始電位狀態

gpioget gpiochip0 1

```
root@romulus:~# gpioget gpiochip0 1
0
```

圖 3: GPIOA1 初始值為 0

✓ 完成:GPIO 環境確認完成, GPIOA1 可用於監控服務開發。

Part 2: 程式碼開發

步驟 4: 建立 C++ 主程式

目的: 使用 libgpiod API 開發 GPIO 監控服務

gpio-monitor.cpp (輪詢模式實作):

```
#include <gpiod.hpp>
#include <iostream>
#include <chrono>
#include <thread>
#include <string>

// GPIO Configuration
constexpr auto gpioChip = "gpiochip0";
constexpr auto gpioLine = 1; // GPIOA1
constexpr auto gpioName = "GPIOA1";
constexpr auto pollIntervalMs = 1000; // Poll every 1 second

int readGpioValue()
{
    // Open chip and get line within this scope
    // GPIO is released when function returns
    gpiod::chip chip(gpioChip);
    auto line = chip.get_line(gpioLine);

    gpiod::line_request config;
    config.consumer = "gpio-monitor";
    config.request_type = gpiod::line_request::DIRECTION_INPUT;

    line.request(config);
    int value = line.get_value();
    // line is automatically released when it goes out of scope

    return value;
}
```

圖 4-1: 程式碼 - 標頭檔與 readGpioValue 函數

```
int main()
{
    std::cout << "[GPIO Monitor] Starting GPIO monitor service (polling mode)..."
               << std::endl;
    std::cout << "[GPIO Monitor] Monitoring: " << gpioName
               << " (chip: " << gpioChip << ", line: " << gpioLine << ")"
               << std::endl;
    std::cout << "[GPIO Monitor] Poll interval: " << pollIntervalMs << "ms"
               << std::endl;

    try
    {
        // Get initial value
        int lastValue = readGpioValue();
        std::cout << "[GPIO Monitor] Initial value: " << lastValue << std::endl;
        std::cout << "[GPIO Monitor] Monitoring for state changes..." << std::endl;
        std::cout << "[GPIO Monitor] (Use 'gpioset gpiochip0 1=1' or '1=0' to test)"
                  << std::endl;

        int pollCount = 0;
```

圖 4-2: 程式碼 - main 函數初始化

```

// Main polling loop
while (true)
{
    std::this_thread::sleep_for(std::chrono::milliseconds(pollIntervalMs));

    int currentValue = readGpioValue();
    pollCount++;

    if (currentValue != lastValue)
    {
        std::string eventType;
        if (currentValue > lastValue)
        {
            eventType = "RISING_EDGE (0 -> 1)";
        }
        else
        {
            eventType = "FALLING_EDGE (1 -> 0)";
        }

        std::cout << "[GPIO Monitor] *** State Change Detected! ***" << std::endl;
        std::cout << "[GPIO Monitor]   Type: " << eventType << std::endl;
        std::cout << "[GPIO Monitor]   Previous value: " << lastValue << std::endl;
        std::cout << "[GPIO Monitor]   Current value: " << currentValue << std::endl;

        lastValue = currentValue;
    }
    else
    {
        // Every 5 polls, show status
        if (pollCount % 5 == 0)
        {
            std::cout << "[GPIO Monitor] Polling... (current value: "
                << currentValue << ")" << std::endl;
        }
    }
}

```

圖 4-3: 程式碼 - 主迴圈與狀態變化偵測

```

catch (const std::exception& e)
{
    std::cerr << "[GPIO Monitor] Exception: " << e.what() << std::endl;
    return 1;
}

return 0;
}

```


圖 4-4: 程式碼 - 例外處理

步驟 5: 建立 Meson 編譯設定

目的: 設定 C++23 標準和 libgpiod 依賴

```

project(
    'gpio-monitor',
    'cpp',
    version: '1.0',
    default_options: [
        'cpp_std=c++23',
        'warning_level=3',
    ],
)

# Find libgpiod dependency
gpiodcxx = dependency('libgpiodcxx', required: true)

# Build the executable
executable(
    'gpio-monitor',
    'gpio-monitor.cpp',
    dependencies: [gpiodcxx],
    install: true,
    install_dir: '/usr/bin',
)

```

圖 5: meson.build 設定檔

步驟 6: 建立 systemd service

目的: 讓服務開機自動啟動

```

[Unit]
Description=GPIO Monitor Service - Monitor GPIOA1 state changes
After=multi-user.target

[Service]
Type=simple
ExecStart=/usr/bin/gpio-monitor
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

圖 6: systemd service 檔案

步驟 7: 建立 BitBake recipe

目的: 將服務整合到 OpenBMC 編譯系統

```
SUMMARY = "GPIO Monitor Service"
DESCRIPTION = "Monitor GPIOA1 and output state change messages"
LICENSE = "Apache-2.0"
LIC_FILES_CHKSUM = "file://${COREBASE}/meta/files/common-licenses/Apache-2.0;md5=89aea4e17d99a7cacadbeed46a0096b10"

SRC_URI = "file://gpio-monitor.cpp \
           file://meson.build \
           file://gpio-monitor.service"

S = "${WORKDIR}/sources-unpack"

inherit meson pkgconfig systemd

DEPENDS = "libgpiod"

SYSTEMD_SERVICE:${PN} = "gpio-monitor.service"
SYSTEMD_AUTO_ENABLE = "enable"

do_unpack:append() {
    import shutil
    import os

    workdir = d.getVar('WORKDIR')
    s = d.getVar('S')

    os.makedirs(s, exist_ok=True)

    for f in ['gpio-monitor.cpp', 'meson.build', 'gpio-monitor.service']:
        src = os.path.join(workdir, f)
        dst = os.path.join(s, f)
        if os.path.exists(src):
            shutil.copy2(src, dst)
    }

do_install:append() {
    install -d ${D}${systemd_system_unitdir}
    install -m 0644 ${S}/gpio-monitor.service ${D}${systemd_system_unitdir}/
}
```

圖 7: BitBake recipe 檔案

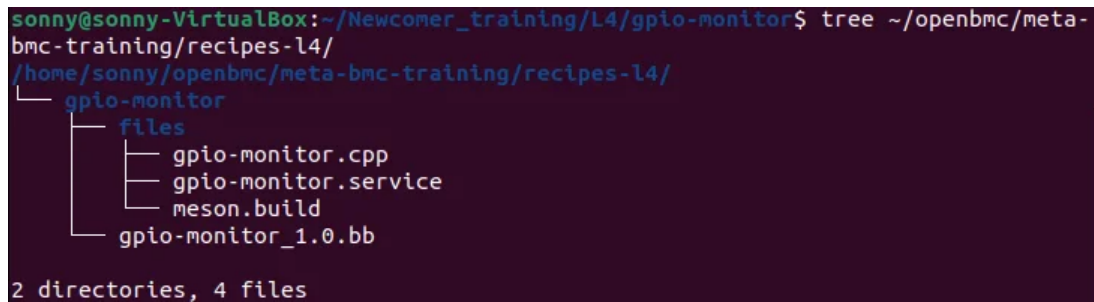
✓ 完成: 程式碼開發完成, 包含主程式、編譯設定、服務檔和 recipe。

Part 3: 編譯與部署

步驟 8: 建立 meta-layer 結構

目的: 將服務放入 meta-bmc-training layer

```
mkdir -p ~/openbmc/meta-bmc-training/recipes-l4/gpio-monitor/files
cp gpio-monitor.cpp meson.build gpio-monitor.service files/
cp gpio-monitor_1.0.bb ../
```



```
sonny@sonny-VirtualBox:~/Newcomer_training/L4/gpio-monitor$ tree ~/openbmc/meta-bmc-training/recipes-l4/
/home/sonny/openbmc/meta-bmc-training/recipes-l4/
├── gpio-monitor
│   ├── files
│   │   ├── gpio-monitor.cpp
│   │   ├── gpio-monitor.service
│   │   └── meson.build
│   └── gpio-monitor_1.0.bb
2 directories, 4 files
```

圖 8: meta-layer 結構

步驟 9: 編譯服務

目的: 使用 BitBake 編譯 GPIO 監控服務

```
cd ~/openbmc
source setup romulus
bitbake gpio-monitor
```

編譯成功後使用 SCP 傳送到 QEMU 進行測試:

```
scp -P 2222 ../gpio-monitor root@127.0.0.1:/usr/bin/
```

✓ 完成: 編譯成功, 服務已部署到 QEMU。

Part 4: 測試與驗證

步驟 10: 事件模式測試(遇到問題)

目的:測試使用 EVENT_BOTH_EDGES 模式的監控服務

初始版本使用事件模式(EVENT_BOTH_EDGES)，但遇到「Device or resource busy」錯誤：

```

sonny@sonny-VirtualBox: ~ - ssh - p 2222 root@127.0.0.1
Starting Wait for /xyz/openbmc_project/control/host0/restriction_mode...
Starting Wait for /xyz/openbmc_project/state/bmc0...
Starting Wait for /xyz/openbmc_project/state/host0...
Starting Wait for /xyz/openbmc_project/tlne/sync_method...
[ OK ] Started Phosphor Hwmon Poller.
[ OK ] Started Phosphor Hwmon Poller.
Starting Phosphor Version Software Manager...
Starting Phosphor Chassis Data Manager...
[ OK ] Started Hostname Service.
[ OK ] Finished Phosphor Sysfs - Add LED.
[ OK ] Finished Phosphor Sysfs - Add LED.
[ OK ] Finished Phosphor Sysfs - Add LED.
[ OK ] Finished Wait for /org/openbmc/control/power0.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/auto_reboot.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/boot_one_time.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/boot.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/power_restore_policy.
[ OK ] Finished Wait for /xyz/openbmc_project/control/host0/restriction_mode.
[ OK ] Finished Wait for /xyz/openbmc_project/tlne/sync_method.

Phosphor OpenBMC (Phosphor OpenBMC Project Reference Distro) nodistro.0 romulus
ttyS4
romulus login: sonny@sonny-VirtualBox: ~ - ssh - p 2222 root@127.0.0.1
sonny@sonny-VirtualBox: ~ - ssh - p 2222 root@127.0.0.1
root@127.0.0.1's password:
root@romulus:~# gpiochip0
gpiochip0: error setting the GPIO line values: Device or resource busy
root@romulus:~# gpioinfo gpiochip0 | head -n 5
gpiochip0 - 231 lines:
    line 0:      unnamed      unused      input      active-high
    line 1:  "cfam-reset"      "gpio-monitor" input active-high [used]
    line 2:      unnamed      unused      input      active-high
    line 3:      unnamed      unused      input      active-high
root@romulus:~#

```

圖 9: 事件模式測試 - GPIO 被服務佔用, gpioset 無法改值

備註:事件模式會持續佔用 GPIO, 等待硬體中斷。在 QEMU 模擬環境中無法用 gpioset 模擬外部輸入。

步驟 11: 輪詢模式測試(成功)

目的:改用輪詢模式讓 GPIO 在讀取間隔釋放

修改程式為輪詢模式後，成功偵測 GPIO 狀態變化：

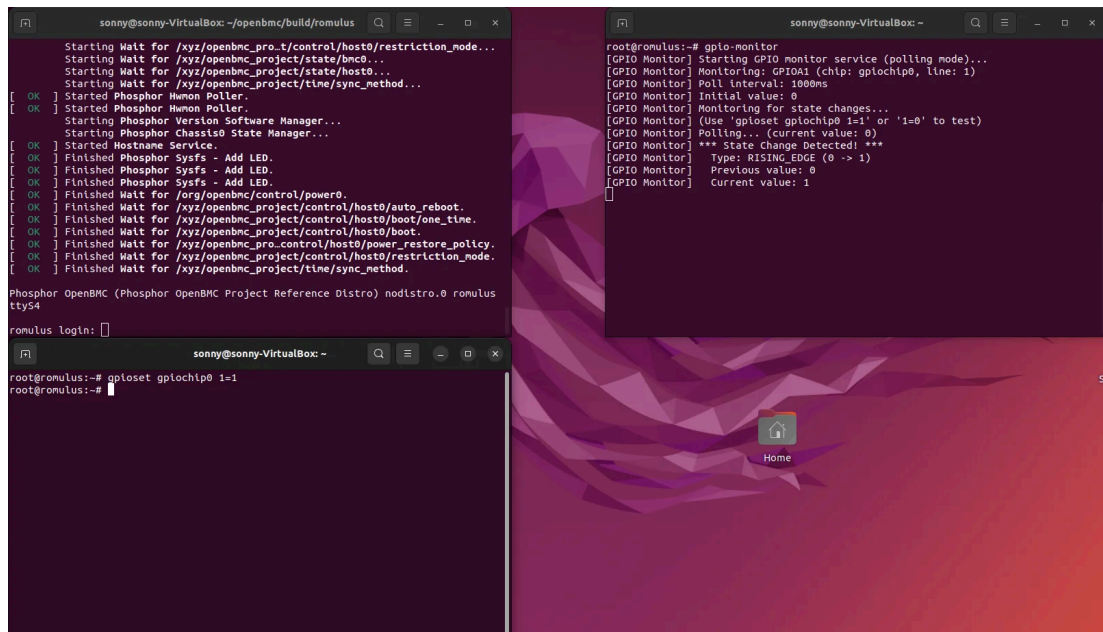


圖 10: 輪詢模式 - 偵測到 *RISING_EDGE*

步驟 12: 完整測試 (上升緣與下降緣)

目的: 驗證服務能偵測雙向狀態變化

```
gpiowrite gpiochip0 1=1 # 觸發 RISING_EDGE
gpiowrite gpiochip0 1=0 # 觸發 FALLING_EDGE
```

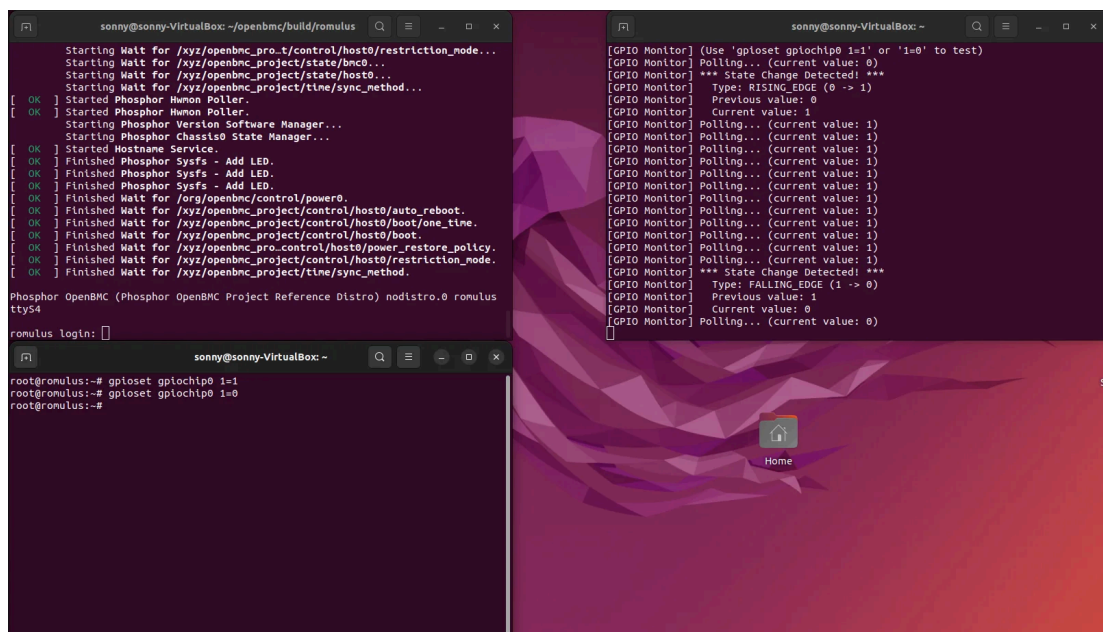


圖 11:完整測試 - RISING EDGE 與 FALLING EDGE 皆成功偵測

✓ 完成:GPIO 監控服務功能驗證完成！

技術困難與解決方案

遇到的問題

使用 gpioset 測試時出現 "Device or resource busy" 錯誤。

原因分析

libgpiod 的事件模式 (EVENT_BOTH_EDGES) 會持續獨佔 GPIO，等待硬體中斷觸發。由於 QEMU 模擬環境沒有實體外部輸入，無法產生硬體中斷，而 gpioset 也無法 request 已被佔用的 GPIO。

解決方案

改用輪詢模式 (Polling Mode)，讓 GPIO 在每次讀取後釋放，使 gpioset 可以在間隔中改變 GPIO 值。

兩種模式比較

模式	GPIO 佔用	觸發來源	適用環境
事件模式	持續佔用	硬體中斷	真實硬體
輪詢模式	間歇佔用	軟體改值	QEMU 測試

結論

完成項目

項目	狀態
GPIO 環境確認	✓ 完成
C++ 程式開發 (libgpod)	✓ 完成
meson.build 編譯設定	✓ 完成
systemd service 檔案	✓ 完成
BitBake recipe	✓ 完成
QEMU 測試驗證	✓ 完成
GitHub 上傳	✓ 完成

學習心得

本次 Lab 學習了 GPIO 在 BMC 中的配置與監控方式。主要收穫包括：

1. 了解 AST2500 GPIO 控制器架構，以及 GPIO line 編號與實體腳位的對應關係。
2. 學習使用 libgpod API (gpod::chip、gpod::line、line_request) 進行 GPIO 操作。
3. 掌握 gpodetect、gpoinfo、gpoget、gpohset 等除錯工具的使用方法。
4. 理解事件模式與輪詢模式的差異，以及各自適用的場景。
5. 解決 QEMU 環境下 GPIO 測試的限制，成功驗證服務功能。